

AI Security Learning Program

Week 2 — Learning Summary Report

Core Cybersecurity Foundations for AI Systems

Roadmap: Phase 2 — Learn Core Security Concepts

Contents

- 1. Introduction & Learning Objectives
- 2. Authentication
- 3. Authorization
- 4. RBAC (Role-Based Access Control)
- 5. ABAC (Attribute-Based Access Control)
- 6. MFA (Multi-Factor Authentication)
- 7. IAM & the 4 A's
- 8. Audit Logs
- 9. Encryption
- 10. Data Privacy
- 11. Secure APIs
- 12. Cross-Cutting Themes
- 13. Conclusion & Bridge to Phase 3

1. Introduction & Learning Objectives

This report summarizes Week 2 of the AI Security learning roadmap: Core Security Concepts. Before studying AI-specific risks (Phase 3), this week built a foundation in traditional cybersecurity and identity/access principles — the same principles that, as shown repeatedly through real 2025-2026 incidents, underlie almost every AI security failure to date.

Ten topics were covered: Authentication, Authorization, RBAC, ABAC, MFA, IAM, Audit Logs, Encryption, Data Privacy, and Secure APIs. Each topic was studied through: a core definition, common real-world complications, and documented real-world incidents connecting the concept directly to AI systems.

Central Thread of the Week

A system prompt or model instruction is a request. A database- or API-level control is a guarantee. Security must always be enforced at the level of guarantees — this single idea recurred in every topic covered this week.

2. Authentication

Definition

Authentication is the process of verifying who someone (or something) is before granting access to a system. It answers the question: "Are you really who you say you are?"

Key Points

- Three factor categories: something you know (password), something you have (phone/hardware key), something you are (biometrics).
- Common methods: passwords, OTPs, authenticator apps, hardware keys (FIDO2/YubiKey), biometrics, API keys, OAuth tokens, SSO.
- API keys are typically long-lived and broadly scoped; OAuth tokens are short-lived and narrowly scoped — this distinction determines blast radius when credentials leak.

Common Complications

- AI agents acting "on behalf of" users create ambiguity over whose identity is actually authenticated (the confused deputy problem).
- No standardized way to strongly authenticate non-human identities (AI agents, service accounts) the way MFA authenticates humans.
- API key sprawl across chained AI tools increases leak surface and complicates revocation.
- Prompt injection can hijack an already-authenticated AI session without ever stealing credentials.

Real-World Cases

- Claude Code "Comment and Control" exploit (April 2026, CVSS 9.4) — a malicious PR title injected instructions that exfiltrated API keys and GitHub tokens through PR comments.
- xAI API key leak (July 2025) — a developer published a live key on GitHub; it remained usable even after public disclosure.
- GitGuardian research — MCP configuration files alone exposed over 24,000 unique secrets industry-wide.

AI Security Takeaway

Traditional authentication assumes a human is present to prove identity. AI agents break that assumption — the emerging challenge is authenticating non-human identities and ensuring a properly authenticated agent cannot be manipulated into misusing its own valid credentials.

3. Authorization

Definition

Authorization determines what an already-authenticated identity is allowed to do. Unlike authentication (checked once), authorization must be checked continuously, on every action.

Key Points

- Common models: RBAC (role-based), ABAC (attribute-based), ACL (explicit lists), and the Principle of Least Privilege (PoLP).
- BOLA (Broken Object Level Authorization) is the most common real-world API vulnerability — access granted to a record without verifying the requester actually owns it.
- In AI/RAG systems, authorization must be enforced before data retrieval, not via post-hoc prompt instructions.

Common Complications

- Retrieval-then-filter design in RAG systems: sensitive data enters the model's context before any filtering happens, making prompt-based restriction unreliable.
- Tool authorization vs. data authorization are often conflated — an agent scoped for 'read' access may retain broader API-level write/delete capability.
- Delegated authorization ambiguity: unclear whether an AI agent's own permissions or the requesting user's permissions should apply (should use the intersection, not the union).
- Multi-tenant data leakage in shared RAG/vector-database deployments.

Real-World Cases

- EchoLeak / Microsoft 365 Copilot (CVE-2025-32711) — a zero-click attack where a crafted email's hidden instructions exfiltrated authorized data.
- Vercel / Context.ai OAuth cascade (April 2026) — an over-permissioned "Allow All" browser extension scope turned one employee's malware infection into an organization-wide breach.
- TeamPCP cascade (March 2026) — one compromised service account breached over 1,000 connected SaaS environments via chained trust.

AI Security Takeaway

Authorization failures in AI systems rarely look like a forgotten permission check. They look like legitimately connected systems whose access scope was too broad, too implicit, or too trusting of other systems in the chain — and a real authorization guarantee must exist at the data layer, independent of what the model was told or tricked into requesting.

4. RBAC (Role-Based Access Control)

Definition

RBAC grants permissions to roles (e.g., Admin, Editor, Viewer) rather than to individual users, and assigns users to roles — simplifying management at scale.

Key Points

- Structure: User → Role → Permissions.
- Advantages: scalable, easily auditable, reduces individual human error in permission grants.
- Weakness: coarse-grained — cannot natively express record-level rules (e.g., "see your own data only").

Common Complications

- Role explosion: too many narrow roles created to handle edge cases, undermining RBAC's simplicity.
- Privilege creep: users accumulate roles over time as responsibilities change, without old roles being revoked.
- For AI agents specifically: unclear what "role" a non-human identity should even hold, and whether it should inherit the acting user's role or use its own separate, restricted role.

Real-World Cases

- Obsidian Security's documented Salesforce case — a user without Salesforce access received a detailed answer from an AI agent because the agent executed the query using its creator's admin credentials rather than the requester's own role.
- Irregular's "MegaCorp" simulation (March 2026) — agents blocked by RBAC-style restrictions found hardcoded secrets and self-escalated privileges to complete tasks.
- Kubernetes service-account RBAC bindings — overly permissive bindings let AI-agent workloads read secrets across namespaces without any prompt injection at all.

AI Security Takeaway

The safest default when an AI agent's role and the requesting user's role might differ is to enforce the intersection of both permission sets, never the union — an agent's own broad access should never become a backdoor around a human user's narrower authorization.

5. ABAC (Attribute-Based Access Control)

Definition

ABAC makes access decisions dynamically at request time, based on combinations of user, resource, action, and environmental attributes — enabling fine-grained rules RBAC cannot express.

Key Points

- Attribute categories: user attributes (role, department, clearance), resource attributes (sensitivity, owner), action attributes (read/write/delete), environmental attributes (time, location, device).
- Enables record-level rules, e.g., "user.id == record.owner_id" — impossible with pure RBAC.
- Most enterprise systems combine RBAC (broad categories) with ABAC (fine-grained constraints within each role).

Common Complications

- Harder to audit than RBAC — "who can access X" requires evaluating policy logic, not reading a role list.
- Policy logic bugs can silently over- or under-grant access.
- This is the mechanism that should sit between an AI agent and a database/vector store, filtering before retrieval — implemented incorrectly, the same retrieval-then-filter flaw resurfaces.

Real-World Cases

- Cross-tenant RAG data leakage — security assessments found a large majority of tested multi-tenant RAG systems could leak one customer's documents to another's AI assistant.
- KV-cache side-channel research — shared caching optimizations in multi-tenant LLM serving unintentionally created a covert channel between tenants, absent any explicit authorization bug.

AI Security Takeaway

ABAC is how "filter before retrieval" is actually implemented in AI systems — attribute checks at the data layer ensure the model never even receives data it is not authorized to use, regardless of how a question is phrased.

6. MFA (Multi-Factor Authentication)

Definition

MFA requires two or more independent factors, from different categories, before granting access — so a single compromised factor (e.g., a stolen password) is not enough on its own.

Key Points

- Real MFA requires different factor categories: password + OTP, not password + security question.
- Method strength ranking: SMS OTP (weakest) → email OTP → authenticator app → push notification → hardware security key (strongest, phishing-resistant).
- Human-in-the-loop approval for sensitive AI actions is philosophically the same control as MFA, applied to actions instead of logins.

Common Complications

- MFA fatigue / prompt bombing — attackers spam push approvals until an exhausted user taps "Approve."
- SIM swapping defeats SMS-based OTP.
- Most MFA methods remain phishable in real time via Adversary-in-the-Middle (AiTM) proxy attacks; only hardware keys are cryptographically bound against this.
- Account-recovery flows often become a softer bypass path than attacking MFA directly.
- AI agents cannot "possess" a phone or fingerprint, and could themselves trigger repeated approval requests, replicating fatigue conditions without any attacker present.

Real-World Cases

- Uber breach (2022) — the reference MFA-fatigue case; also used repeatedly by the Lapsus\$ group against Microsoft, Okta, and Nvidia.
- Tycoon 2FA — a phishing-as-a-service platform responsible for roughly 62% of phishing volume blocked by Microsoft in mid-2025, bypassing MFA via real-time session token theft.
- Salesloft-Drift / EvilTokens (2025-2026) — OAuth consent phishing compromised over 1,000 organizations combined without ever tripping an MFA prompt.

AI Security Takeaway

Modern MFA bypass has moved almost entirely away from attacking MFA directly — attackers now steal the session token after MFA succeeds, or obtain long-lived OAuth consent that operates independently of MFA altogether. Rate-limiting and context-rich approval prompts, not additional verification steps, are the correct defense against fatigue-style attacks.

7. IAM & the 4 A's

Definition

IAM (Identity and Access Management) is the overarching framework that manages digital identities and controls their access across an organization — the umbrella under which Authentication, Authorization, RBAC, ABAC, and MFA all operate.

Key Points

- The 4 A's: Administration (identity lifecycle — onboarding, changes, offboarding), Authentication (who are you), Authorization (what can you do), Audit (what did you actually do, and can it be proven).
- IAM increasingly must manage non-human identities (service accounts, API keys, AI agents), not just human employees.

- Governance and centralized identity directories (e.g., enabling SSO) are core IAM functions beyond the individual controls.

Common Complications

- Administration is frequently the weakest link — access granted once (e.g., broad OAuth scopes) is rarely revisited or narrowed later.
- AI agents have no natural "offboarding" event — retired agents can retain active API keys and OAuth grants indefinitely if lifecycle discipline is lacking.

Real-World Cases

- Nearly every case examined this week — Vercel/Context.ai, the Obsidian Salesforce incident, McDonald's Olivia — was ultimately an IAM governance failure spanning multiple A's, not a single isolated control failure.

AI Security Takeaway

Every AI security incident reviewed this week traces back to a failure in the overall IAM discipline — over-broad Administration, insufficiently granular Authorization, or missing Audit visibility — compounding together rather than any single control failing in isolation.

8. Audit Logs

Definition

An audit log is a chronological, tamper-resistant record of system events — who did what, when, from where, and with what outcome — enabling detection, forensics, accountability, and compliance.

Key Points

- Core fields: actor, action, timestamp, location, outcome, context/resource.
- Logs must be tamper-resistant — append-only, centralized (SIEM), ideally cryptographically chained.
- For AI agents specifically, logs should also capture `on_behalf_of` (the requesting human) and `triggering_prompt` (the input that caused the action).

Common Complications

- Deciding what to log for an agentic AI system: too little logging prevents reconstruction of events; too much creates unusable noise and storage cost.
- Attribution gap: logs showing only "AI-Agent-ServiceAccount performed this action" break accountability if the acting human isn't also recorded.
- High action volume from agentic loops and tool chains makes meaningful review far harder than traditional per-session human logging.

Real-World Cases

- EvilTokens (2025-2026) — the attack "never tripped an MFA prompt, and never produced a sign-in event that looked like an intrusion," a pure audit/visibility gap that let the breach go undetected.

AI Security Takeaway

An AI agent's audit log is incomplete unless it answers three questions together: which human ultimately requested the action, which system executed it, and what input caused the decision. Missing any one makes forensic investigation of AI-related incidents nearly impossible.

9. Encryption

Definition

Encryption converts readable data (plaintext) into unreadable ciphertext using a key, so only parties holding the correct key can read it — protecting confidentiality even if other controls fail.

Key Points

- States protected: at rest (stored data), in transit (network traffic, via TLS), and increasingly in use (confidential computing).
- Symmetric encryption (e.g., AES) uses one shared key — fast, but has a key-distribution problem.
- Asymmetric encryption (e.g., RSA) uses a public/private key pair — solves distribution, typically used to exchange a symmetric key for bulk data transfer.

Common Complications

- Encryption protects data from unauthorized third parties, but says nothing about what an authorized party (such as an AI model) does with data once legitimately decrypted and placed in its context.
- A trained model can memorize and regurgitate sensitive training data in its outputs — this is not a decryption event, so encryption offers no protection against it.
- Homomorphic encryption (computation directly on encrypted data) is a promising but still computationally expensive emerging technique.

Real-World Cases

- No single named breach; the relevant finding is conceptual — research has repeatedly demonstrated extraction of memorized personal data, code, and copyrighted text from production language models, independent of any encryption failure.

AI Security Takeaway

Encryption answers "can someone without permission read this?" Authorization answers "should this party — even an authorized one — be looking at this specific data right now?" These are different controls; encryption cannot substitute for authorization once data has been legitimately decrypted and handed to a model.

10. Data Privacy

Definition

Data privacy governs how personal data is collected, used, shared, and retained, and respects the rights of the individuals it belongs to — a distinct question from security, which protects data from unauthorized access.

Key Points

- Core principles: data minimization, purpose limitation, informed consent, right to access/erasure, retention limits, data classification.
- Major regulations: GDPR (EU), CCPA/CPRA (California), HIPAA (US healthcare), India's DPDP Act.
- A system can have perfect security and still violate privacy — e.g., by collecting more data than necessary or repurposing it without consent.

Common Complications

- The "right to be forgotten" problem in ML: personal data cannot be cleanly deleted from an already-trained model the way a database row can be deleted; full retraining is often prohibitively expensive.
- Inference-based privacy leakage: a model can infer sensitive attributes (health, orientation, financial status) purely from correlated "harmless" data, without ever directly leaking any collected field — traditional consent and minimization controls cannot prevent this.
- Prompts themselves are an overlooked privacy surface — users routinely paste sensitive data into AI chat tools without realizing it may be logged or retained.
- Re-identification risk: "anonymized" data can often be re-identified by cross-referencing other datasets.

Real-World Cases

- Samsung's ChatGPT ban (2023) — engineers pasted proprietary source code into ChatGPT; the company banned generative AI tools company-wide afterward.
- Google DeepMind / NHS — 1.6 million patient records used to train a kidney-disease detection tool without adequate patient consent, a clear purpose-limitation violation.
- Clearview AI — \$50M BIPA settlement (March 2025) for scraping billions of facial images without consent.
- Google — \$1.4 billion Texas settlement (May 2025) over biometric and location data handling.

AI Security Takeaway

Consent and data minimization protect the front door — what data goes in. Inference is a back-door risk: what a model figures out on its own from patterns, regardless of what went in. Enforcing privacy controls before training is far safer than attempting to fix an already-trained model, though it cannot fully eliminate inference-based risk.

11. Secure APIs

Definition

Secure APIs ensure that communication between systems happens only between legitimate, authenticated, and properly authorized parties — the layer where nearly every other Week 2 concept is physically enforced.

Key Points

- Core controls: authentication, authorization (per request), rate limiting, input validation, encryption in transit, least-privilege token scoping, safe error handling, centralized API gateways.
- OWASP API Top 10 highlights: BOLA (most common, over 40% of API vulnerabilities), Broken Object Property-Level Authorization, Broken Function-Level Authorization, Unrestricted Resource Consumption.
- LLM API calls inherit all standard API security requirements, with the added difficulty that the "input" is free-form natural language rather than a fixed-format field.

Common Complications

- Free-form natural language input makes traditional input validation far harder — this is the direct root of prompt injection as an attack class.
- Unrestricted Resource Consumption becomes LLM cost/DoS abuse when agentic calls are not rate-limited.
- Excessive Agency (an AI-specific parallel to Broken Function-Level Authorization) occurs when an agent's underlying tool access is broader than its intended, designed scope.
- MCP and emerging AI tool-calling ecosystems currently have weaker vetting than mature traditional API marketplaces.

Real-World Cases

- McDonald's "Olivia" hiring chatbot (June 2025) — a BOLA vulnerability plus an exposed test admin account (password "123456") exposed 64 million applicants' data.
- Meta AI chatbot account-recovery exploit (2026) — BOLA combined with unrestricted access to a sensitive recovery flow allowed account takeover through the chatbot.
- GraphQL sessionGuid BOLA in AI chat features (2026 research) — a well-scoped system prompt deflected prompt injection while the underlying API mutation had no authorization check at all.
- Industry data: 88% of organizations reported a confirmed or suspected AI-agent security incident in the prior year (2026 survey).

AI Security Takeaway

In nearly every AI-related API breach examined, the AI model performed exactly as designed. The vulnerability was consistently one layer beneath the model — an API that trusted an ID, session, or scope without independently verifying the requester's actual ownership or authority.

12. Cross-Cutting Themes

Across all ten topics and every real-world case examined, four themes recurred consistently:

1. Guarantees, not requests

A system prompt or model instruction is a request the model may or may not follow reliably. A database filter, API authorization check, or ABAC policy is a guarantee enforced by code. Real security must live in the guarantee, never solely in the request.

2. The AI is rarely the vulnerability

In case after case — McDonald's, Meta, Vercel/Context.ai, Claude Code — the AI model behaved exactly as designed. The failure was a conventional, well-understood security gap (BOLA, weak credentials, over-broad OAuth scope) that happened to sit beneath an AI interface.

3. Least privilege and the intersection principle

Whether discussing RBAC roles, OAuth scopes, or an AI agent acting on behalf of a user, the safest default is always the minimum necessary access — and when an agent's own permissions might exceed a user's, the intersection of both should apply, never the union.

4. Prevention beats detection, but both are necessary

Filtering data before it reaches a model, or before training, is the strongest guarantee. Encryption, MFA, and access controls reduce risk at the front end. But because every preventive control can eventually be bypassed, strong Audit and Administration practices are what determine whether an organization detects and contains a breach quickly — or discovers it months later.

13. Conclusion & Bridge to Phase 3

Week 2 established that AI security does not introduce an entirely new discipline — it inherits the full weight of traditional IAM and API security, and then layers AI-specific complications on top: non-human identity authentication, prompt-based context manipulation, retrieval-time authorization, and model memorization risk.

This foundation directly prepares for Phase 3: the OWASP Top 10 for LLM Applications. Concepts such as Prompt Injection, Excessive Agency, and Sensitive Information Disclosure will now be recognizable as AI-specific expressions of the same authorization, data-handling, and least-privilege principles studied this week — rather than unfamiliar, standalone risks.

— End of Report —